Write a C program to simulate page replacement algorithms.

A) FIFO

b) LRU

c) Optimal

```
#include <stdio.h> #include<stdbool.h>

#define MAX_FRAMES 10
#define MAX_REF 50

void printFrames(int frames[], int num_frames) {
for (int i = 0; i < num_frames; i++) {
        if (frames[i] == -1) printf("[ ] ");
        else
            printf("[%d] ", frames[i]);
    }
    printf("\n");
}

void fifo(int ref[], int n, int num_frames) { int
    frames[MAX_FRAMES];
    int page_faults = 0;
    int index = 0;

    for (int i = 0; i < num_frames; i++) frames[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");

    for (int i = 0; i < n; i++) {
    int page = ref[i];
        bool hit = false;

        for (int j = 0; j < num_frames; j++) {
        if (frames[j] == page) {
                hit = true;
                break;
            }
        }

        printf("Ref %d: ", page); if (!hit) {
            frames[index] = page;
            index = (index + 1) % num_frames; page_faults++;
```

```
            printFrames(frames, num_frames);
        } else {
            printf("Hit -> ");
            printFrames(frames, num_frames);
        }
    }
    printf("Total Page Faults: %d\n", page_faults);
    printf("Total Page Hits: %d\n", n - page_faults);
}

void optimal(int ref[], int n, int num_frames) {
int frames[MAX_FRAMES];
    int page_faults = 0;

    for (int i = 0; i < num_frames; i++) frames[i] = -1;

    printf("\n--- Optimal Page Replacement ---\n");

    for (int i = 0; i < n; i++) {
    int page = ref[i];
        bool hit = false;

        for (int j = 0; j < num_frames; j++) {
        if (frames[j] == page) {
                hit = true; break;
            }
        }

        printf("Ref %d: ", page); if (!hit) {
            bool empty_space = false;
            for (int j = 0; j < num_frames; j++) {
            if (frames[j] == -1) {
                    frames[j] = page;
                    empty_space = true;
                    break;
                }
            }

            if (!empty_space) {
                int victim_index = -1;
                int farthest = i;

                for (int j = 0; j < num_frames; j++) {
                int next_use = n;
                    for (int k = i + 1; k < n; k++) {
                    if (frames[j] == ref[k]) {
```

```
                                next_use = k;
                                break;
                        }
                    }
                    if (next_use > farthest) { farthest =
                        next_use; victim_index = j;
                    }
                }
                frames[victim_index] = page;
            }
            page_faults++;
            printFrames(frames, num_frames);
        } else {
            printf("Hit -> ");
            printFrames(frames, num_frames);
        }
    }
    printf("Total Page Faults: %d\n", page_faults);
    printf("Total Page Hits: %d\n", n - page_faults);
}

void lru(int ref[], int n, int num_frames) {
int frames[MAX_FRAMES];
    int last_used[MAX_FRAMES];
    int page_faults = 0;

    for (int i = 0; i < num_frames; i++) { frames[i] = -1;
        last_used[i] = -1;
    }

    printf("\n--- LRU Page Replacement ---\n");

    for (int i = 0; i < n; i++) {
    int page = ref[i];
        bool hit = false; int
        hit_index = -1;

        for (int j = 0; j < num_frames; j++) {
        if (frames[j] == page) {
                hit = true;
                hit_index = j;
                break;
            }
        }

        printf("Ref %d: ", page);
```

```
            if (!hit) {
                bool empty_space = false;
                for (int j = 0; j < num_frames; j++) { if (frames[j]
                    == -1) {
                        frames[j] = page;
                        last_used[j] = i;
                        empty_space = true; break;
                    }
                }

                if (!empty_space) {
                    int victim_index = 0;
                    int min_time = last_used[0];

                    for (int j = 1; j < num_frames; j++) { if
                        (last_used[j] < min_time) {
                            min_time = last_used[j];
                            victim_index = j;
                        }
                    }
                    frames[victim_index] = page;
                    last_used[victim_index] = i;
                }
                page_faults++; printFrames(frames,
                num_frames);
            } else {
                last_used[hit_index] = i; printf("Hit -> ");
                printFrames(frames, num_frames);
            }
        }
        printf("Total Page Faults: %d\n", page_faults); printf("Total Page
        Hits: %d\n", n - page_faults);
}

int main() {
        int num_frames, n; int
        ref[MAX_REF];

        printf("Enter number of frames: ");
        scanf("%d", &num_frames);

        printf("Enter length of reference string: ");
        scanf("%d", &n);

        printf("Enter the reference string space-separated:\n");
        for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &ref[i]);
    }

    fifo(ref, n, num_frames);
    optimal(ref, n, num_frames);
    lru(ref, n, num_frames);

    return 0;
}
```

Output:

```
Enter number of frames: 3
Enter length of reference string: 7
Enter the reference string space-separated:
1 3 0 3 5 6 3

--- FIFO Page Replacement ---
Ref 1: [1] [ ] [ ]
Ref 3: [1] [3] [ ]
Ref 0: [1] [3] [0]
Ref 3: Hit -> [1] [3] [0]
Ref 5: [5] [3] [0]
Ref 6: [5] [6] [0]
Ref 3: [5] [6] [3]
Total Page Faults: 6
Total Page Hits: 1

--- Optimal Page Replacement ---
Ref 1: [1] [ ] [ ]
Ref 3: [1] [3] [ ]
Ref 0: [1] [3] [0]
Ref 3: Hit -> [1] [3] [0]
Ref 5: [5] [3] [0]
Ref 6: [6] [3] [0]
Ref 3: Hit -> [6] [3] [0]
Total Page Faults: 5
Total Page Hits: 2

--- LRU Page Replacement ---
Ref 1: [1] [ ] [ ]
Ref 3: [1] [3] [ ]
Ref 0: [1] [3] [0]
Ref 3: Hit -> [1] [3] [0]
Ref 5: [5] [3] [0]
Ref 6: [5] [3] [6]
Ref 3: Hit -> [5] [3] [6]
Total Page Faults: 5
Total Page Hits: 2
```